

Final Project: Comprehensive Database Security and Management

Lab Report

SecureShop — Database Security and Management

Date: July 11, 2026

Overview

This project implements and secures a relational database for SecureShop, an online retailer, using MySQL and phpMyAdmin. The SecureShop database includes four related tables — customers, orders, payment_details, and products — and this report documents the four tasks completed for the project: importing and exploring the database, applying encryption and data masking to protect sensitive customer information, managing user privileges for a Delivery Executive role, and demonstrating and analyzing a SQL injection vulnerability in an unsanitized query.

Environment	Online MySQL lab environment (phpMyAdmin)
Tools Used	phpMyAdmin, MySQL 8.0 (SQL Query editor), AES_ENCRYPT / AES_DECRYPT, MySQL user and privilege management (CREATE USER, GRANT, REVOKE, SHOW GRANTS)
Tasks Completed	4 of 4 (database setup and exploration, data encryption and masking, user privilege management, SQL injection demonstration)

Task 1: Database Setup and Exploration

Objective: Import the SecureShop database into the MySQL environment and explore its tables using SQL queries.

1.1 Import the Database

I imported the SecureShop.sql dump file into phpMyAdmin, creating the SecureShop database with four tables: customers, orders, payment_details, and products. The Structure view confirms all four tables were imported successfully, each containing 10 rows.

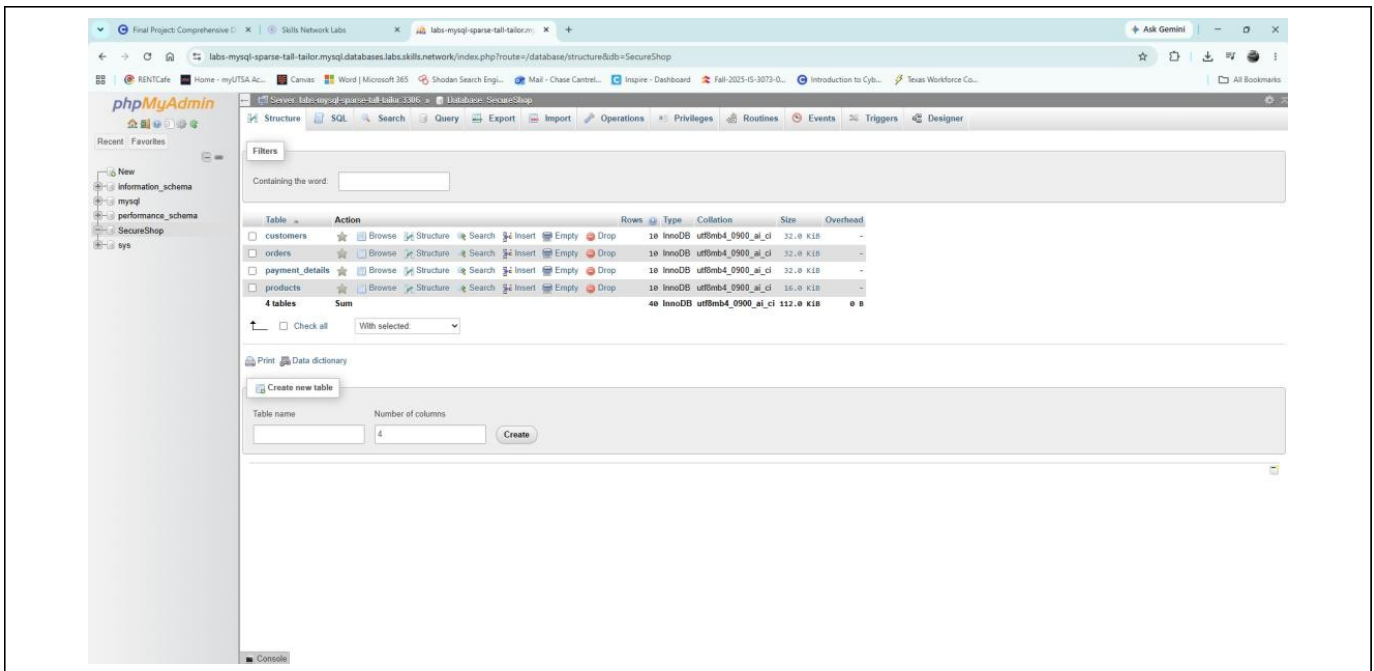


Figure 1. phpMyAdmin Structure view of the SecureShop database, showing the four imported tables: customers, orders, payment_details, and products.

1.2.1 Explore the payment_details Table

I ran `SELECT * FROM payment_details;` to view the full contents of the payment_details table, confirming the payment_id, order_id, payment_date, payment_method, and payment_amount columns for all 10 payment records.

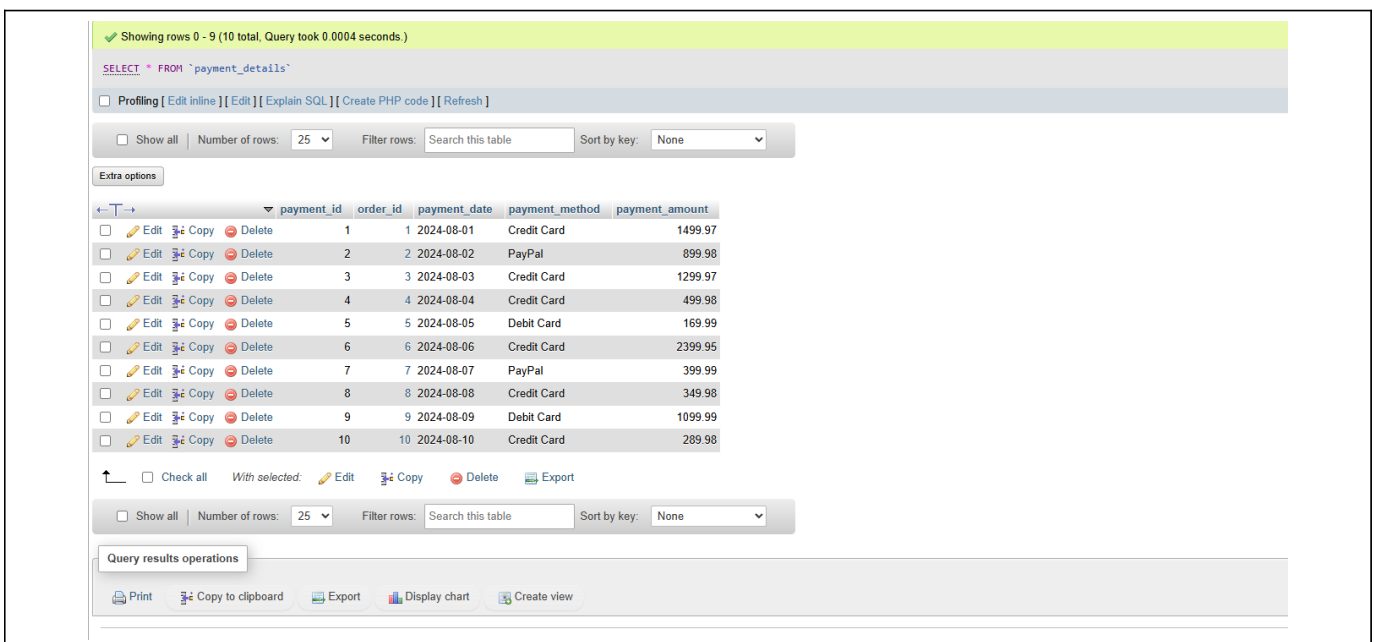


Figure 2. Query output listing all 10 rows of the payment_details table.

1.2.2 Search for All Customers

I ran `SELECT * FROM customers;` to retrieve every record in the customers table, returning all 10 customers with their contact and address details.

Showing rows 0 - 9 (10 total, Query took 0.0005 seconds.)

SELECT * FROM customers;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	customer_id	first_name	last_name	email	phone	address	city	state	postal_code	country
☐ Edit Copy Delete	1	John	Doe	john.doe@example.com	123-456-7890	123 Elm St	New York	NY	10001	USA
☐ Edit Copy Delete	2	Jane	Smith	jane.smith@example.com	987-654-3210	456 Oak St	Los Angeles	CA	90001	USA
☐ Edit Copy Delete	3	Michael	Brown	michael.brown@example.com	555-123-4567	789 Pine St	Chicago	IL	60601	USA
☐ Edit Copy Delete	4	Emily	Davis	emily.davis@example.com	444-555-6666	101 Maple St	Houston	TX	77001	USA
☐ Edit Copy Delete	5	David	Wilson	david.wilson@example.com	333-444-5555	202 Birch St	Phoenix	AZ	85001	USA
☐ Edit Copy Delete	6	Sarah	Johnson	sarah.johnson@example.com	111-222-3333	303 Cedar St	Philadelphia	PA	19101	USA
☐ Edit Copy Delete	7	James	Williams	james.williams@example.com	666-777-8888	404 Spruce St	San Antonio	TX	78201	USA
☐ Edit Copy Delete	8	Olivia	Martinez	olivia.martinez@example.com	999-000-1111	505 Cherry St	San Diego	CA	92101	USA
☐ Edit Copy Delete	9	William	Garcia	william.garcia@example.com	222-333-4444	606 Aspen St	Dallas	TX	75201	USA
☐ Edit Copy Delete	10	Isabella	Rodriguez	isabella.rodriguez@example.com	777-888-9999	707 Redwood St	San Jose	CA	95101	USA

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Figure 3. Query output showing all customer records retrieved from the customers table.

1.2.3 List All Products

I ran `SELECT * FROM products;` to list every product available in the store, confirming all 10 products with their descriptions, prices, and stock quantities.

Showing rows 0 - 9 (10 total, Query took 0.0005 seconds.)

SELECT * FROM products;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	product_id	product_name	product_description	price	stock_quantity
☐ Edit Copy Delete	1	Laptop	High-performance laptop	999.99	50
☐ Edit Copy Delete	2	Smartphone	Latest model smartphone	699.99	100
☐ Edit Copy Delete	3	Tablet	10-inch screen tablet	299.99	75
☐ Edit Copy Delete	4	Smartwatch	Smartwatch with fitness tracking	199.99	200
☐ Edit Copy Delete	5	Headphones	Noise-cancelling headphones	149.99	150
☐ Edit Copy Delete	6	Camera	Digital camera with 4K video	499.99	30
☐ Edit Copy Delete	7	Printer	Wireless color printer	129.99	40
☐ Edit Copy Delete	8	Monitor	27-inch 4K monitor	399.99	25
☐ Edit Copy Delete	9	Keyboard	Mechanical keyboard	89.99	80
☐ Edit Copy Delete	10	Mouse	Wireless ergonomic mouse	49.99	100

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Figure 4. Query output listing all 10 products from the products table.

1.2.4 Retrieve Payment Details for Order ID 7

I ran `SELECT * FROM payment_details WHERE order_id = 7;` to isolate the payment record tied to order 7. The result shows a single row: payment_id 7, paid by PayPal on 2024-08-07 for \$399.99.

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds)

```
SELECT * FROM payment_details WHERE order_id = 7;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	payment_id	order_id	payment_date	payment_method	payment_amount
☐ Edit ☐ Copy ☐ Delete	7	7	2024-08-07	PayPal	399.99

Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Figure 5. Query output showing the payment details for order_id = 7.

1.2.5 Orders Placed by the Customer Named David

I ran `SELECT * FROM customers WHERE first_name = 'David';` which correctly identified David Wilson (customer_id 5) but returned only his customer record rather than his order history. To fully answer the task, this query needs to be followed by a join against the orders table on customer_id — for example, `SELECT o.* FROM orders o JOIN customers c ON o.customer_id = c.customer_id WHERE c.first_name = 'David';` — which, based on the SecureShop data, returns David Wilson's single order: order_id 5, placed on 2024-08-05, for a total of \$169.99.

Showing rows 0 - 0 (1 total, Query took 0.0006 seconds)

```
SELECT * FROM customers WHERE first_name = 'David';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	customer_id	first_name	last_name	email	phone	address	city	state	postal_code	country
☐ Edit ☐ Copy ☐ Delete	5	David	Wilson	david.wilson@example.com	333-444-5555	202 Birch St	Phoenix	AZ	85001	USA

Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

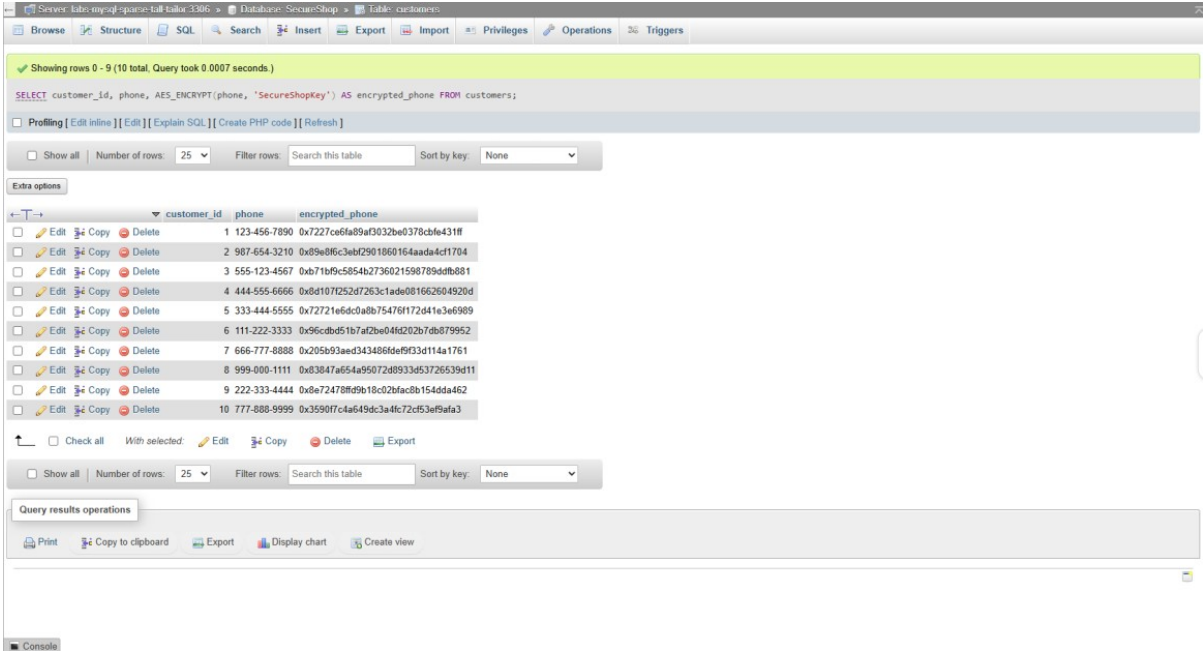
Figure 6. Query output identifying David Wilson in the customers table. This captures the customer lookup; the corresponding order-history join (order_id 5, 2024-08-05, \$169.99) was identified from the schema but was not captured in a separate screenshot.

Task 2: Data Encryption and Masking

Objective: Protect sensitive customer data at rest by encrypting phone numbers with AES and masking postal codes so that only partial values are exposed.

2.1.1 Encrypt Customer Phone Numbers

I used `AES_ENCRYPT(phone, 'SecureShopKey')` AS `encrypted_phone` to generate an encrypted version of each customer's phone number alongside the original plain-text value, confirming the encryption function produces a distinct hexadecimal ciphertext for every row.



Showing rows 0 - 9 (10 total, Query took 0.0007 seconds)

```
SELECT customer_id, phone, AES_ENCRYPT(phone, 'SecureShopKey') AS encrypted_phone FROM customers;
```

customer_id	phone	encrypted_phone
1	123-456-7890	0x7227ce6fa89af3032be0378cbfe431ff
2	987-654-3210	0xb9e8bfc3ebf2901860164aada4cf1704
3	555-123-4567	0xb71b9c5854b2736021598709d0fb881
4	444-555-6666	0xd107f252d7263c1ade08166260492bd
5	333-444-5555	0x72721e6dc0a8b754781172441e3e6989
6	111-222-3333	0x96cbbd51b7af2be04fd02b7db679952
7	666-777-8888	0x205b93aed3434866de9f33d114a1761
8	999-000-1111	0xd3847a654a95072d8933d53726539d11
9	222-333-4444	0x8e72478fd9b18c02bfac0b154dda462
10	777-888-9999	0x35907c4a649dc3a4fc72cd53e9afa3

Figure 7. Query output showing each customer's original phone number next to its AES-encrypted value.

2.1.2 Decrypt Customer Phone Numbers

To confirm the encryption was reversible with the correct key, I nested `AES_ENCRYPT` inside `AES_DECRYPT` using the same key, 'SecureShopKey', and cast the result back to `CHAR`. The output shows each `customer_id` alongside its correctly decrypted phone number, matching the original values.

Showing rows 0 - 9 (10 total. Query took 0.0006 seconds.)

```
SELECT customer_id, CAST(AES_DECRYPT( AES_ENCRYPT(phone, 'SecureShopKey'), 'SecureShopKey') AS CHAR) AS decrypted_phone FROM customers;
```

Query results table:

customer_id	decrypted_phone
1	123-456-7890
2	987-654-3210
3	555-123-4567
4	444-555-6666
5	333-444-5555
6	111-222-3333
7	666-777-8888
8	999-000-1111
9	222-333-4444
10	777-888-9999

Figure 8. Query output showing customer phone numbers successfully decrypted back to their original format.

2.2 Mask Customer Postal Codes

I used `CONCAT('*****', SUBSTRING(postal_code, LENGTH(postal_code) - 1, 2))` AS `masked_postal_code` to replace all but the last two digits of each postal code with asterisks. The output confirms every `masked_postal_code` value exposes only the final two digits (for example, `*****01`), protecting the rest of the value.

Showing rows 0 - 9 (10 total. Query took 0.0008 seconds.)

```
SELECT customer_id, postal_code, CONCAT('*****', SUBSTRING(postal_code, LENGTH(postal_code)-1, 2)) AS masked_postal_code FROM customers;
```

Query results table:

customer_id	postal_code	masked_postal_code
1	10001	*****01
2	90001	*****01
3	60601	*****01
4	77001	*****01
5	85001	*****01
6	19101	*****01
7	78201	*****01
8	92101	*****01
9	75201	*****01
10	95101	*****01

Figure 9. Query output showing customer postal codes masked so that only the last two digits remain visible.

Task 3: User Privilege Management

Objective: Create a restricted database user for a Delivery Executive role, grant it the minimum privileges needed on the customers table, and then verify and tighten those privileges.

3.1.1 Create the Delivery Executive User

I ran `CREATE USER 'DeliveryExecutive'@'%' IDENTIFIED BY 'Password123!';` to create a new MySQL user. The command completed with an empty result set and no errors, confirming the user was created successfully.



```

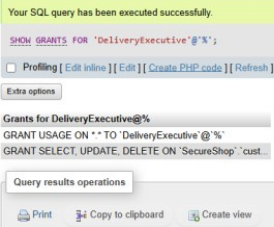
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0145 seconds.)
CREATE USER 'DeliveryExecutive'@'%' IDENTIFIED BY 'Password123!';
[ Edit inline ] [ Edit ] [ Create PHP code ]

```

Figure 10. Confirmation that the CREATE USER command for DeliveryExecutive executed successfully.

3.1.2 Grant SELECT, UPDATE, and DELETE Privileges

I granted the Delivery Executive user the ability to SELECT, UPDATE, and DELETE records in the customers table, then ran SHOW GRANTS FOR 'DeliveryExecutive'@'%'; to verify. The output confirms two grants: general USAGE, and SELECT, UPDATE, DELETE privileges scoped specifically to SecureShop.customers.



```

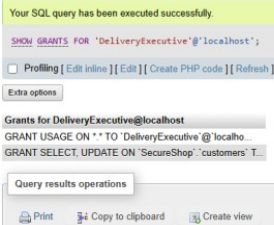
Your SQL query has been executed successfully.
SHOW GRANTS FOR 'DeliveryExecutive'@'%';
[ Profiling ] [ Edit inline ] [ Edit ] [ Create PHP code ] [ Refresh ]
Extra options
Grants for DeliveryExecutive@%
GRANT USAGE ON *.* TO 'DeliveryExecutive'@'%'
GRANT SELECT, UPDATE, DELETE ON 'SecureShop'.customers TO 'DeliveryExecutive'@'%'
Query results operations
[ Print ] [ Copy to clipboard ] [ Create view ]

```

Figure 11. SHOW GRANTS output confirming SELECT, UPDATE, and DELETE privileges granted to DeliveryExecutive on the customers table.

3.1.3 Revoke DELETE Privilege and Verify

Recognizing that DELETE access on customer records was excessive for a delivery role, I revoked it with REVOKE DELETE ON SecureShop.customers FROM 'DeliveryExecutive'@'%';. Re-running SHOW GRANTS confirms the DELETE privilege is no longer listed — only SELECT and UPDATE remain on the customers table.



```

Your SQL query has been executed successfully.
SHOW GRANTS FOR 'DeliveryExecutive'@'localhost';
[ Profiling ] [ Edit inline ] [ Edit ] [ Create PHP code ] [ Refresh ]
Extra options
Grants for DeliveryExecutive@localhost
GRANT USAGE ON *.* TO 'DeliveryExecutive'@'localhost'
GRANT SELECT, UPDATE ON 'SecureShop'.customers TO 'DeliveryExecutive'@'localhost'
Query results operations
[ Print ] [ Copy to clipboard ] [ Create view ]

```

Figure 12. SHOW GRANTS output confirming DELETE has been revoked, leaving only SELECT and UPDATE on the customers table.

Task 4: SQL Injection Vulnerability Demonstration

Objective: Build a query that joins the customers and orders tables by order_id, then demonstrate how failing to sanitize input allows a SQL injection attack to retrieve unauthorized data.

4.1 Join Customers and Orders by Order ID

I built the query SELECT c.*, o.* FROM customers c JOIN orders o ON c.customer_id = o.customer_id WHERE o.order_id = 7; which directly concatenates the order_id value into the WHERE clause without validation or parameterization. Run as intended, it correctly returns the single customer and order record associated with order_id 7 (James Williams).

Showing rows 0 - 0 (1 total. Query took 0.0006 seconds.)

```
SELECT c.*, o.* FROM customers c JOIN orders o ON c.customer_id = o.customer_id WHERE o.order_id = 7;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

customer_id	first_name	last_name	email	phone	address	city	state	postal_code	country	order_id	customer_id	order_date	total_amount
7	James	Williams	james.williams@example.com	666-777-8888	404 Spruce St	San Antonio	TX	78201	USA	7	7	2024-08-07	399.99

Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print | Copy to clipboard | Export | Display chart | Create view

Figure 13. Join query output returning the customer and order details for order_id = 7, using an unsanitized WHERE clause.

4.2 Demonstrate the SQL Injection

Because the query builds its WHERE clause from unvalidated input, I injected the payload `o.order_id = 7 OR 1=1` in place of a normal order_id value. The `1=1` condition is always true, so it overrides the intended filter entirely. Instead of returning one record for order 7, the query returned all 10 joined customer-order records — exposing every customer's name, email, phone, and address alongside every order, confirming the query is vulnerable to SQL injection and can leak unauthorized data.

Showing rows 0 - 9 (10 total. Query took 0.0008 seconds.)

```
SELECT c.*, o.* FROM customers c JOIN orders o ON c.customer_id = o.customer_id WHERE o.order_id = 7 OR 1=1;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

customer_id	first_name	last_name	email	phone	address	city	state	postal_code	country	order_id	customer_id	order_date	total_amount
1	John	Doe	john.doe@example.com	123-456-7890	123 Elm St	New York	NY	10001	USA	1	1	2024-08-01	1499.97
2	Jane	Smith	jane.smith@example.com	987-654-3210	456 Oak St	Los Angeles	CA	90001	USA	2	2	2024-08-02	899.98
3	Michael	Brown	michael.brown@example.com	555-123-4567	789 Pine St	Chicago	IL	60601	USA	3	3	2024-08-03	1299.97
4	Emily	Davis	emily.davis@example.com	444-555-6666	101 Maple St	Houston	TX	77001	USA	4	4	2024-08-04	499.98
5	David	Wilson	david.wilson@example.com	333-444-5555	202 Birch St	Phoenix	AZ	85001	USA	5	5	2024-08-05	169.99
6	Sarah	Johnson	sarah.johnson@example.com	111-222-3333	303 Cedar St	Philadelphia	PA	19101	USA	6	6	2024-08-06	2399.95
7	James	Williams	james.williams@example.com	666-777-8888	404 Spruce St	San Antonio	TX	78201	USA	7	7	2024-08-07	399.99
8	Olivia	Martinez	olivia.martinez@example.com	999-000-1111	505 Cherry St	San Diego	CA	92101	USA	8	8	2024-08-08	349.98
9	William	Garcia	william.garcia@example.com	222-333-4444	606 Aspen St	Dallas	TX	75201	USA	9	9	2024-08-09	1099.99
10	Isabella	Rodriguez	isabella.rodriguez@example.com	777-888-9999	707 Redwood St	San Jose	CA	95101	USA	10	10	2024-08-10	289.98

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print | Copy to clipboard | Export | Display chart | Create view

Figure 14. SQL injection payload (`OR 1=1`) applied to the vulnerable join query, returning all customer and order records instead of the single intended row.

Conclusion

All four tasks for the SecureShop database security and management project were completed. I imported the SecureShop database and explored its customers, orders, payment_details, and products tables with targeted SELECT queries; applied AES encryption and decryption to protect customer phone numbers and used string masking to conceal all but the last two digits of postal codes; created a Delivery Executive database user, granted it SELECT, UPDATE, and DELETE privileges on the customers table, and then revoked the excessive DELETE privilege after verifying it with SHOW GRANTS; and built a join query between customers and orders that was vulnerable to SQL injection, then used an `OR 1=1` payload to demonstrate how unsanitized input can expose data far beyond what a query was intended to return. Together, these tasks demonstrate core database security practices, including data encryption and masking, least-privilege access control, and recognizing and validating against SQL injection vulnerabilities.